Parte A. Contrato del equipo y decisiones de diseño

Proyecto: Lab 2 --- MSG JavaFX
Rol: Controlador (`feature/controller`)
Ubicación sugerida: `docs/parte-a-contrato-y-decisiones.md`

> **Propósito:** registrar la propuesta del contrato compartido y el borrador de respuestas D1--D6. El equipo debe confirmar o ajustar los valores en la Parte A. El Integrador copia el contrato acordado a `BITACORA.md`, sección «Contrato».

A1. Contrato compartido

La tabla fija los nombres y ubicaciones que deben coincidir entre la vista, el controlador, el formato del dinero y la aplicación. Un valor solo se cambia si los cuatro integrantes lo acuerdan y la modificación queda registrada en `BITACORA.md`.

Elemento Valor propuesto Quiénes lo usan

Archivo de la vista
`src/main/resources/co/edu/unillanos/msg/infrastructure/ui/funds-view.fxml` Vista, Integrador

Controlador declarado
`fx:controller="co.edu.unillanos.msg.infrastructure.ui.FundsController"` Vista, Controlador
en la vista

Etiqueta del total Un `Label` con `fx:id="totalLabel"` Vista, Controlador

Acción del botón `onAction="#onEstimate"` Vista, Controlador

Archivo del controlador
`src/main/java/co/edu/unillanos/msg/infrastructure/ui/FundsController.java` Controlador, Integrador

Paquete del controlador `co.edu.unillanos.msg.infrastructure.ui` Controlador, Vista

Constructor del `FundsController(EstimateFundsForWeek estimateFunds)` Controlador, Integrador
controlador

Formato del dinero `MoneyFormat.format(double)` devuelve, por ejemplo, `$ 1160.00` Formato y estilo,
Controlador

Hoja de estilos `funds-view.css`, con las clases `title` y `total` Vista, Formato y estilo

Roles y ramas

Rol Rama(s) Qué escribe o integra Cómo lo demuestra Revisa el PR
de

Integrador `feature/javafx-setup` y Configuración de JavaFX La ventana abre con Controlador;
después `feature/app` en `pom.xml`, `Launcher` `mvn javafx:run` y fusiona los PR
y `App`; integra los PR desde el IDE

Vista `feature/view` `funds-view.fxml` La ventana muestra Formato y
el título, el botón estilo
y la etiqueta con
los nombres del
contrato

Controlador `feature/controller` `FundsController.java` Al pulsar el botón Integrador
se muestra
`$ 1160.00`

Formato y `feature/format` `MoneyFormat`, `MoneyFormatTest` Vista

estilo `MoneyFormatTest` y pasa y el total se

`funds-view.css` ve en verde

Si el equipo queda conformado por tres personas, el Integrador asume también el rol de Formato y estilo. Si el equipo es el mismo del Laboratorio 1, la asignación propuesta es: Hipotecas → Integrador; Inversiones → Vista; Registro → Controlador; Integrador → Formato y estilo. Si el equipo cambió, los roles se pueden elegir libremente, pero el Integrador no puede ser quien ya desempeñó ese rol.

Orden de integración

1. `feature/javafx-setup`: el Integrador la abre al inicio de la Parte B.
2. `feature/format`: no depende de las otras ramas y el controlador utiliza `MoneyFormat`.
3. `feature/view` y `feature/controller`: se pueden integrar en cualquier orden.
4. `feature/app`: se integra al final, porque `App` necesita la vista y el controlador.

Después de cada fusión, quienes sigan trabajando deben traer los cambios de `main` a su rama:

```
None
git fetch origin
git merge origin/main
```

Confirmación del equipo:\

- \(\) Contrato confirmado sin cambios.\
- \(\) Contrato ajustado y cambios registrados en `BITACORA.md`.

Cambios acordados (si los hay):

A2. Borrador de respuestas D1--D6

Estas respuestas son una base para la discusión y deben revisarse con la guía. La Parte A se completa según las indicaciones del docente.

D1. ¿`FundsController` es el Controller de GRASP?

El Controller de GRASP es `EstimateFundsForWeek`: recibe la operación del sistema y coordina el trabajo. `FundsController` es un adaptador de JavaFX que traduce el clic en una llamada al caso de uso y el `double` en texto. Si se confunden, la lógica de negocio termina en la ventana, que es precisamente el error señalado para la Parte D.

D2. ¿Por qué el controlador recibe el caso de uso por el constructor?

Si `FundsController` hiciera `new EstimateFundsForWeek(...)` con sus propios repositorios, la ventana podría utilizar datos distintos a los de `App`/`Main`, por ejemplo repositorios vacíos. Además, cambiar a H2 exigiría tocar el controlador y sería más difícil probarlo con repositorios falsos. Recibir el caso de uso por el constructor evita que el controlador cree y administre esas dependencias.

D3. ¿Por qué el formato del dinero debe estar fuera de `domain`?

El formato `$ 1160.00` ---símbolo monetario, dos decimales y punto decimal--- es una decisión de presentación. Las reglas de negocio deben trabajar con valores numéricos. Si el formato estuviera en `domain`, esa capa quedaría acoplada a la manera en que una pantalla muestra el dinero.

D4. ¿Por qué se deben acordar los nombres del contrato?

Si cada integrante elige sus propios nombres, el FXML podría declarar `fx:id="resultado"` mientras el controlador espera `totalLabel`. La asociación entre la vista y el controlador fallaría al cargar la ventana y podría producir una excepción como `LoadException`. Acordar los nombres antes de programar evita ese error de integración.

D5. ¿Cuál es el orden de integración y por qué?

El orden propuesto es:

`feature/javafx-setup` → `feature/format` → `feature/view` y `feature/controller` (en cualquier orden) → `feature/app`.

Todo archivo que importe clases de `javafx` solo compila cuando JavaFX está configurado en `pom.xml`. El controlador usa `MoneyFormat`, por eso conviene integrar primero `feature/format`. Finalmente, `App` necesita que la vista y el controlador estén disponibles.

D6. ¿Qué archivos de `domain` o `application` se predice que cambiarán?

La predicción es que **ningún archivo de `domain` ni de `application` cambiará**. La lista prevista de archivos modificados en esas dos capas es vacía, porque el trabajo del controlador consiste en adaptar los eventos de la interfaz a una llamada al caso de uso existente, sin trasladar ni modificar la lógica de negocio.

Alcance del rol Controlador

El trabajo de este rol se limita a `FundsController.java` en la rama `feature/controller`. No se deben modificar `domain`, `application`, `pom.xml`, `App`, el FXML, el CSS ni `MoneyFormat`, salvo que el equipo acuerde formalmente un cambio de contrato y lo registre en `BITACORA.md`.

El controlador debe recibir `EstimateFundsForWeek` por su constructor, invocar `estimateFunds.execute()` desde `onEstimate()`, presentar el resultado mediante `MoneyFormat.format(double)` y manejar la `IllegalStateException` según lo establecido en la guía. No debe implementar fórmulas ni crear casos de uso o repositorios propios.